CTU

CZECH TECHNICAL UNIVERSITY IN PRAGUE

F3

Faculty of Electrical Engineering
Department of Computer Science

Bachelor's Thesis

Web-based application for managing private bank accounts

Yelyzaveta Taranova
Software Engineering and Technology

Leden 2013, srpen 2020
Supervisor: Ondřej Vondrouš

Acknowledgement / Declaration

Chtěl bych poděkovat své manželce Ludmile za podporu nejen finanční. Díky tomu mohu na svém pracovišti dělat, co mě baví, a nejsem stresován výplatní páskou.

I declare that I have prepared the submitted thesis independently and that I have cited all the information sources used in accordance with the Methodological Guideline on the observance of ethical principles in the preparation of university final theses.

In Prague 13. 13. 2013 ???

.....

Abstrakt / Abstract

Tento projekt se zaměřuje na návrh a vývoj webové aplikace pro správu osobních bankovních účtů prostřednictvím agregace finančních údajů z více českých bank na jednom místě. Jeho hlavním cílem je poskytnout uživatelům jednotný, přehledný a uživatelsky přívětivý náhled na jejich finance, aniž by museli přepínat mezi různými systémy internetového bankovnictví.

Pro určení struktury a funkčnosti aplikace projekt začíná analýzou stávajících platforem pro správu osobních financí. Tato analýza identifikuje běžné funkce, silné stránky a omezení současných řešení a slouží jako základ pro návrh klíčových funkcí navrhované aplikace.

Klíčová slova: Bankovní aplikace, osobní bankovnictví, centralizace financí, webová aplikace, Spring Boot Framework, REST API, Java, službově orientovaná architektura.

This project focuses on the design and development of a web application for managing personal bank accounts by aggregating financial data from multiple Czech banks in one place. Its primary purpose is to provide users with a unified, clear, and user-friendly overview of their finances without requiring them to switch between different online banking systems.

To define the structure and functionality of the application, the project begins with an analysis of existing personal finance platforms. This analysis identifies common features, strengths, and limitations of current solutions and serves as the basis for designing the core functions of the proposed application.

Keywords: Banking application, personal banking, finance centralization, web application, Spring Boot Framework, REST API, Java, service-oriented architecture.

Contents /

1 Introduction **1**
1.1 Motivation 1
1.2 Objectives 1

2 Existing Solutions Overview **3**
2.1 Budgeting Applications 3
2.2 Wealth Trackers 3
2.3 Global Solutions with European Support 4
2.4 Summary of the Market Gap . . 4
2.5 Key Features in Existing Finance Applications 4

3 Czech Banking APIs Overview **6**
3.1 The Payment Services Directive (PSD2) 6
3.2 Authentication and Security Protocols 6
3.3 Sandbox vs. Production Environments 6
3.4 Comparison of Czech Banking APIs 7
3.4.1 Provided Features 7
3.4.2 Production Environment Requirements 7
3.4.3 Sandbox Environment Requirements 7
3.5 Conclusion 7

4 Business Analysis **8**
4.1 Functional Requirements 8
4.1.1 Access to Public Pages . . . 8
4.1.2 Profile Management 8
4.1.3 Bank Integration 9
4.1.4 Financial Data Retrieval . . 9
4.1.5 Visualization 10
4.1.6 Categorization 10
4.1.7 Transactions Grouping . . 10
4.1.8 Product Filtering 11
4.1.9 Budgeting 11
4.1.10 System Monitoring 11
4.2 Non-Functional Requirements . 11
4.3 Use cases 12
4.3.1 Anonymous User 12
4.3.2 Logged-in User 12
4.4 Process Analysis 14
4.4.1 Connecting a Bank Account 14

5 Technology Analysis **16**
5.1 Backend Development 16
5.1.1 Framework 16
5.1.2 Programming Language . . 16
5.1.3 Database 16
5.1.4 Caching 16
5.1.5 Resilience 17
5.1.6 Messaging Configuration . 17
5.1.7 Identity Provider 17
5.1.8 Conclusion 17
5.2 Frontend Development 18
5.2.1 Framework 18

6 System Design **19**
6.1 Architecture 19
6.1.1 Architectural Style 19
6.1.2 Architecture Diagram . . . 19
6.1.3 Internal Architecture . . . 20
6.1.4 Component Diagram 21
6.2 Data design 23
6.2.1 Database Model 23
6.2.2 Domain Class Model 26

7 Implementation **30**
7.1 Security 30
7.1.1 Keycloak 30
7.2 Features 30
7.2.1 Dashboard 30
7.2.2 Banks Overview 31
7.2.3 Accounts Overview 32
7.2.4 Transactions Overview . . . 33
7.2.5 Budgeting 33
7.3 Caching and Resilience 33
7.4 Deployment 33
7.4.1 CI Pipeline 33
7.4.2 Docker 33
7.5 Error Handling 33

8 Conclusion **34**

References **35**

A Zadání práce **37**

B Zkratky a symboly **38**
B.1 Zkratky 38
B.2 Symboly 38
B.3 Soubory, které jsou součástí CTUstyle3 39

Tables / Figures

4.1 Functional Requirements — Access to Public Pages 8
4.2 Functional Requirements — Profile Management 8
4.3 Functional Requirements — Bank Integration 9
4.4 Functional Requirements — Financial Data Retrieval 9
4.5 Functional Requirements — Visualization 10
4.6 Functional Requirements — Categorization 10
4.7 Functional Requirements — Transactions Grouping 10
4.8 Functional Requirements — Product Filtering 11
4.9 Functional Requirements — Budgeting 11
4.10 Functional Requirements — System monitoring 11
4.11 Non-functional Requirements .. 11
5.1 Selected technologies for application backend development 17
4.1 Use Cases — Anonymous User 12
?? Use Cases — Logged-in User .. ??
4.3 Process Diagram — Connecting a Bank Account 14
6.1 Application Architecture 20
6.2 Internal Architecture 21
6.3 Component Diagram 22
6.4 Party Schema 24
6.5 Authorized Client Schema 24
6.6 Bank Schema 25
6.7 Product Schema 25
6.8 Categorization Schema 26
6.9 Domain Class Model - Bank Connection Service 27
6.10 Domain Class Model - Product Service 28
6.11 Domain Class Model - Categorization Service 29
7.1 Feature — Dashboard 30
7.2 Feature — Banks Overview 31
7.3 Feature — Accounts Overview 32
7.4 Feature — Transactions Overview 33

Chapter 1
Introduction

Keeping track of personal finances is getting harder since most people now use multiple bank accounts and cards from different institutions. Czech banks generally have good online platforms, but checking your overall financial situation usually means logging into several different apps to piece the information together.

Thanks to PSD2 and open banking rules in the EU, it is now possible for third-party apps to access bank data securely, as long as the user gives permission. This thesis takes advantage of that to build a unified personal finance dashboard.

The goal of this project is to develop a web application that uses these modern banking APIs to pull a user's accounts into one place. Through a single interface, users can check their current balances, track transactions across their various Czech bank accounts, and manage their budgets. The application is designed purely for personal use and focuses on organizing data safely. To keep the project secure and focused, it only reads data and strictly avoids active operations like sending payments or changing account settings.

1.1 Motivation

Even though Czech banks support PSD2 APIs, their systems are still isolated from one another. If a user has accounts across Česká spořitelna, Air Bank, and Moneta, they have to jump between different apps just to see how much money they actually have.

There are international apps that try to solve this, but they usually struggle to connect with Czech banks. When they do work, they often rely on third-party data aggregators, which raises privacy concerns and usually means the user's data is passing through extra hands.

Because of this, the project is motivated by four main goals:

- **Centralization** — pulling all accounts and transaction history into a single application with an easy-to-read dashboard.
- **Transparency** — giving the user a clear, instant view of their cash flow without requiring them to do manual calculations.
- **Control and privacy** — keeping the user's financial data secure and private, without relying on third-party data brokers.
- **Simplicity** — building a straightforward personal tool instead of a massive, complicated fintech product.

1.2 Objectives

The main goal of this thesis is to build a web application that safely connects to different Czech banks and helps the user make sense of their money. Specifically, the project focuses on four main tasks:

1. Build a single dashboard for all accounts

Instead of forcing users to log into several different bank apps, the application will display all current balances and transaction histories in one place.

2. Make spending habits easy to understand Raw lists of transactions are hard to read. The app will categorize expenses and show basic monthly trends so users can quickly see where their money is going without having to use a spreadsheet.

3. Put open banking to practical use Instead of just conducting theoretical research, the project will actively utilize PSD2 APIs from Czech banks to demonstrate how this technology can safely solve a real-world problem for regular users.

4. Learn from existing applications Before designing the app, current personal finance tools will be analyzed. Seeing what works—and what frustrates users—in existing apps will directly shape the features and interface of this project.

Chapter 2
Existing Solutions Overview

Before designing the new application, it is important to analyze how existing tools handle personal finance management. The core features across the industry are generally the same: pulling transaction data, categorizing spending, and providing visual budgeting tools. Applications differ significantly in their technical implementation, specifically in how they access banking data and their geographic focus. This section evaluates several popular solutions to identify their strengths, design patterns, and limitations regarding Czech banking systems.

2.1 Budgeting Applications

Applications built for the US market often set the standard for user interface design and budgeting methodology.

- **You Need A Budget (YNAB)** (https://www.ynab.com/) uses a strict budgeting approach where every unit of income is assigned a specific category before it is spent. From a functional perspective, its budgeting logic is highly effective.
- **Monarch Money** (https://www.monarchmoney.com/) focuses on household finances, allowing multiple users to manage a shared database. It also actively scans transaction histories to flag recurring subscriptions.
- **PocketGuard** (https://pocketguard.com/) calculates the user's exact disposable income. The application subtracts scheduled upcoming expenses from the current account balance, providing a direct metric of the funds currently available for discretionary spending.

Limitations: although functionally advanced, these applications share a major technical flaw for European users. They rely almost exclusively on North American API aggregators like Plaid. Because they lack integration with the European PSD2 framework, they cannot connect to Czech banks.

2.2 Wealth Trackers

- **Empower** (https://www.empower.com/) is designed to track entire financial portfolios. It aggregates daily checking accounts alongside investment portfolios, retirement funds, and real estate values. The application's primary function is to calculate total net worth and analyze long-term asset allocation rather than strictly monitoring daily spending.
- **Quicken Simplifi** (https://www.quicken.com/products/simplifi/) connects to bank accounts, credit cards, and investment accounts to provide real-time updates on cash flow and spending trends. Users can create custom budgets, establish savings goals, and track specific spending categories through personalized watchlists.

Limitations: tracking multiple asset classes requires a complex user interface. Both applications share the same geographic data restrictions. They rely entirely on North American financial aggregators and lack direct API support for Central European financial institutions. Consequently, they cannot connect to Czech banks via open banking APIs.

2.3 Global Solutions with European Support

- **Spendee** (https://www.spendee.com/) is an application with Czech origins that maintains a large global user base. It functions properly within the Czech Republic by offering detailed transaction categorization and handling multiple currencies. It can automatically synchronize with local institutions like Česká spořitelna or Air Bank, providing an automated overview of daily finances for regional users.
- **Wallet** (https://budgetbakers.com/) is another globally recognized personal finance application developed in the Czech Republic. Similar to Spendee, it provides advanced transaction categorization, budgeting tools, and multi-currency support. It also features automatic data synchronization with Czech banks.

Limitations: to support thousands of banks worldwide, neither application connects to financial institutions directly. Instead, they rely on third-party data aggregators like Salt Edge. This approach introduces a middleman into the data flow. The user must trust a third-party broker with their financial consent and data, which creates potential privacy and security concerns compared to direct bank connections.

2.4 Summary of the Market Gap

Analyzing these existing solutions highlights a clear gap that this thesis aims to fill. The applications with the best interfaces (YNAB, Monarch) cannot connect to Czech banks. The applications that can connect to local banks (Spendee, Wallet) rely on third-party data brokers, adding a layer of privacy risk.

This justifies the core technical approach of this project: building a straightforward, localized tool that connects directly to Czech bank APIs. By skipping the third-party aggregators, the application can guaranty that the user's data travels strictly between their bank and their own private dashboard.

2.5 Key Features in Existing Finance Applications

Based on the analysis of existing solutions, standard personal finance applications typically implement a common set of core functionalities:

- **Account Aggregation** — Connecting various financial accounts to consolidate balances and transaction histories into a single interface. The standard approach involves automated, periodic data synchronization via APIs or data brokers.
- **Transaction Categorization** — Automatically importing and assigning metadata (categories such as `Groceries` or `Utilities`) to individual transactions.
- **Budget Management** — Creating spending limits per category on a defined cycle, usually monthly.
- **Goal Tracking** — Defining specific financial targets, such as accumulating an emergency fund or reducing specific debts.

- **Recurring Payment Detection** — Analyzing transaction histories to identify predictable, repeating charges like utility bills or software subscriptions. This feature allows users to anticipate upcoming cash flow requirements and identify redundant services.
- **Data Visualization** — Generating charts and graphs to visually represent financial states. Standard outputs include charts for categorical spending, graphs for cash flow trends, and historical net worth progression.
- **Automated Notifications** — Triggering system alerts based on specific account states, such as approaching a budget limit, identifying a critically low balance, or detecting unusual transaction patterns.
- **Security and Synchronization** — Implementing strong encryption and multi-factor authentication to protect sensitive financial data.

Chapter 3
Czech Banking APIs Overview

Modern personal finance applications require secure integrations with banking systems. In the Czech Republic, major banks expose standardized APIs mandated by European regulations. These interfaces allow third-party providers to access account information following explicit user consent. The APIs generally align with the Czech Open Banking Standard or the Berlin Group framework, providing consistent data structures for retrieving account details, balances, and transaction histories. This section outlines the regulatory foundation, technical capabilities, security requirements, and access conditions for these APIs.

3.1 The Payment Services Directive (PSD2)

The revised Payment Services Directive (PSD2) is a European Union regulatory framework that fundamentally changed how financial data is managed. Prior to this legislation, banks held exclusive control over their customers' account information. PSD2 mandates that all European financial institutions must provide secure, standardized API access to licensed Third-Party Providers.

This regulation established two primary types of access: Payment Initiation Services (PIS) and Account Information Services (AIS). For the scope of this application, only AIS is relevant. Under the AIS framework, a user can grant a third-party application explicit consent to retrieve their account information and transaction history in a read-only format. The directive ensures that users maintain complete ownership of their financial data and strictly control who can access it.

3.2 Authentication and Security Protocols

Czech institutions implement these mandated services using RESTful API architectures. Authentication relies strictly on the OAuth 2.0 protocol combined with Strong Customer Authentication (SCA). During the synchronization process, the application redirects the user to the bank's own secure portal. At this stage, the user must complete the SCA process to formally grant access. This authorization flow ensures that the custom application never handles or stores direct login credentials. All subsequent data exchanges require mutual TLS encryption to guarantee transport security.

3.3 Sandbox vs. Production Environments

Accessing banking APIs depends entirely on the deployment environment. Connecting to production APIs to fetch real user data requires a valid eIDAS certificate and an official license from the Czech National Bank to operate as a regulated Account Information Service Provider. Acquiring this regulatory approval is highly complex and generally not feasible for private tools or academic projects.

To resolve this access limitation, financial institutions provide developer sandbox environments. These sandboxes replicate the exact architecture and endpoint structure of the production APIs and return realistic, mock financial data. Developers can register applications, implement the OAuth authentication flow, and test data retrieval processes without holding a formal PSD2 license.

3.4 Comparison of Czech Banking APIs

To successfully integrate multiple financial institutions into a unified application, it is necessary to evaluate the technical capabilities and access requirements of their respective Application Programming Interfaces (APIs). This section provides a comparative analysis of the major banks used in the Czech Republic: Komerční banka, Česká spořitelna, Fio Bank, Air Bank, Raiffeisenbank, ČSOB, Moneta Bank, and UniCredit.

3.4.1 Provided Features

All mentioned financial institutions fully support the core Account Information Services required for a personal finance dashboard. Specifically, their APIs provide standardized endpoints for **listing personal accounts**, **listing current account balances**, and **listing transaction history**.

3.4.2 Production Environment Requirements

To access the production environments and retrieve real user data, all six institutions enforce strict regulatory and technical prerequisites. Every bank requires the applicant to hold a valid PSD2 AISP license. From a technical perspective, establishing a production connection universally requires a Qualified Website Authentication Certificate (QWAC).

3.4.3 Sandbox Environment Requirements

The requirements for sandbox environments are significantly less restrictive. For all of the evaluated financial institutions, developers simply need to complete a standard registration process on their respective developer portals to obtain sandbox credentials, API keys, and test certificates.

All mentioned banks, except Moneta Money Bank, Fio Bank, and UniCredit, allow independent developers and private applications to access their testing infrastructure. Moneta Money Bank, Fio Bank, and UniCredit Bank prohibit academic projects and private applications from using their testing platforms, making their APIs unavailable for independent development and research.

3.5 Conclusion

The analysis of banking APIs available in the Czech Republic shows that major banks offer highly standardized Account Information Services driven by the PSD2 framework. Access to real production data remains strictly regulated and limited to licensed third-party providers. The sandbox environments offered by these banks significantly lower the access barriers. These simulated environments provide a highly suitable platform for development, prototyping, and fully verifying functional architecture. This project will make use of the accessible sandbox APIs: Komerční banka, Česká spořitelna,Air Bank, Raiffeisenbank and ČSOB.

Chapter 4
Business Analysis

This chapter defines the business context of the application and translates user needs into structured functional and non-functional requirements. It identifies the target audience, describes expected system behavior, and outlines key business processes supported by the application. The goal of this analysis is to ensure that the proposed solution aligns with real user needs while remaining technically feasible and compliant with regulatory constraints.

4.1 Functional Requirements

This section specifies the functional requirements of the application, defining what the system shall do from the user's perspective. The requirements are grouped by functional areas and cover all major interactions, including authentication, bank integration, financial data retrieval, visualization, budgeting, and administration.

4.1.1 Access to Public Pages

ID	Title	Description
FR1	Displaying the landing page	The system shall allow anonymous users to view the landing page with a general description of the application and its purpose, information about features, security, and supported banks.

Table 4.1. Functional Requirements — Access to Public Pages

4.1.2 Profile Management

ID	Title	Description
FR2	Registration	The system shall allow anonymous users to create an account via the registration form.
FR3	Log in	The system shall support logging in using an email and password.
FR4	Log out	The system shall allow the user to log out.

Table 4.2. Functional Requirements — Profile Management

4.1.3 Bank Integration

ID	Title	Description
FR5	Linking a bank	The system shall allow users to link a bank from defined list of financial institutions
FR6	Token storage	The system shall securely store tokens required for retrieving data.
FR7	Bank disconnection	The system shall allow users to disconnect any previously linked bank.

Table 4.3. Functional Requirements — Bank Integration

4.1.4 Financial Data Retrieval

ID	Title	Description
FR8	Product retrieval	The system shall retrieve customers products from all linked banks.
FR9	Account balances retrieval	The system shall retrieve the balance for each account from all linked banks.
FR10	Transaction history retrieval	The system shall retrieve the transaction history for each account from all linked banks for the specified time period.
FR11	Financial data caching	The system shall cache financial data periodically.

Table 4.4. Functional Requirements — Financial Data Retrieval

4.1.5 Visualization

ID	Title	Description
FR12	Displaying a dashboard	The system shall display a consolidated dashboard containing an overview of assets and liabilities.
FR13	Displaying statistics	The system shall present graphs of spending and income for the selected time period.
FR14	Displaying transactions	The system shall display transactions in a clear list format for every account from all linked banks.
FR15	Displaying products	The system shall display products with balances in a clear list format for every account from all linked banks.
FR16	Displaying budgets	The system shall display budgets in a clear list format.
FR17	Displaying profile	The system shall display the profile with user data.

Table 4.5. Functional Requirements — Visualization

4.1.6 Categorization

ID	Title	Description
FR18	Transactions categorization	The system shall automatically categorize transactions using predefined rules.
FR19	Products categorization	The system shall automatically categorize products using predefined rules.

Table 4.6. Functional Requirements — Categorization

4.1.7 Transactions Grouping

ID	Title	Description
FR20	Transactions Grouping	The system shall group transactions by month and by category.

Table 4.7. Functional Requirements — Transaction Grouping

4.1.8 Product Filtering

ID	Title	Description
FR21	Product Filtering	The system shall allow the filtering of products by bank.

Table 4.8. Functional Requirements — Product Filtering

4.1.9 Budgeting

ID	Title	Description
FR22	Setting monthly budgets	The system shall allow users to set monthly budgets for categories.
FR23	Budget warnings	The system shall display warnings or highlights when a budget is exceeded.

Table 4.9. Functional Requirements — Budgeting

4.1.10 System Monitoring

ID	Title	Description
FR24	System logs	The system shall provide logs for authentication attempts, API errors, and system faults.

Table 4.10. Functional Requirements — System monitoring

4.2 Non-Functional Requirements

ID	Title	Description
NFR1	Security	The application must use secure storage and must not store sensitive data in an open form.
NFR2	Resilient system	The system must handle API failures gracefully.
NFR3	Application performance	The application should load the dashboard within 20 seconds.
NFR4	Browser compatibility	The application must function properly on current versions of major web browsers.

Table 4.11. Non-functional Requirements

4.3 Use cases

This section presents the main use cases of the system, describing how different types of users interact with the application. Use cases are divided according to user roles and illustrate the scope of functionality available to each role. The diagrams provide a high-level overview of system behavior and support the understanding of functional requirements.

4.3.1 Anonymous User

Anonymous users are visitors who access the application without creating an account or logging in.

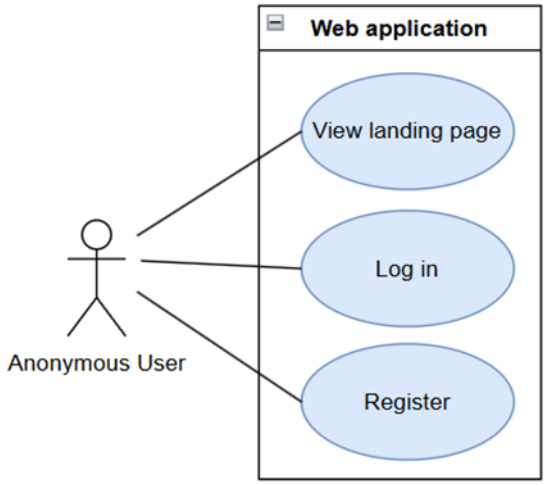

Figure 4.1. Use Cases — Anonymous User

This diagram illustrates the initial interaction points available to an unauthenticated individual, designated as the `Anonymous User` actor. Within the system boundary of the web application, this actor has direct associations with three primary use cases: viewing the public-facing landing page, creating a new account, and authenticating into the system.

These three use cases directly implement the foundational access and authentication functional requirements defined in section 4.1:

- **View landing page** — This use case implements FR1 (Displaying the landing page) from section 4.1.1. It ensures the system allows unauthenticated users to read general information regarding the application's purpose, features, and security.
- **Register** — This use case implements FR2 (Registration) from section 4.1.2. It represents the workflow where an anonymous user submits their credentials via a registration form to create a new profile and gain system access.
- **Log in** — This use case implements FR3 (Log in) from section 4.1.2. It fulfills the requirement for the system to authenticate a user via their credentials, successfully transitioning their state from an Anonymous User to a Logged-in User.

4.3.2 Logged-in User

A logged-in user has access to the core features of the application, including connecting banking accounts and viewing financial data.

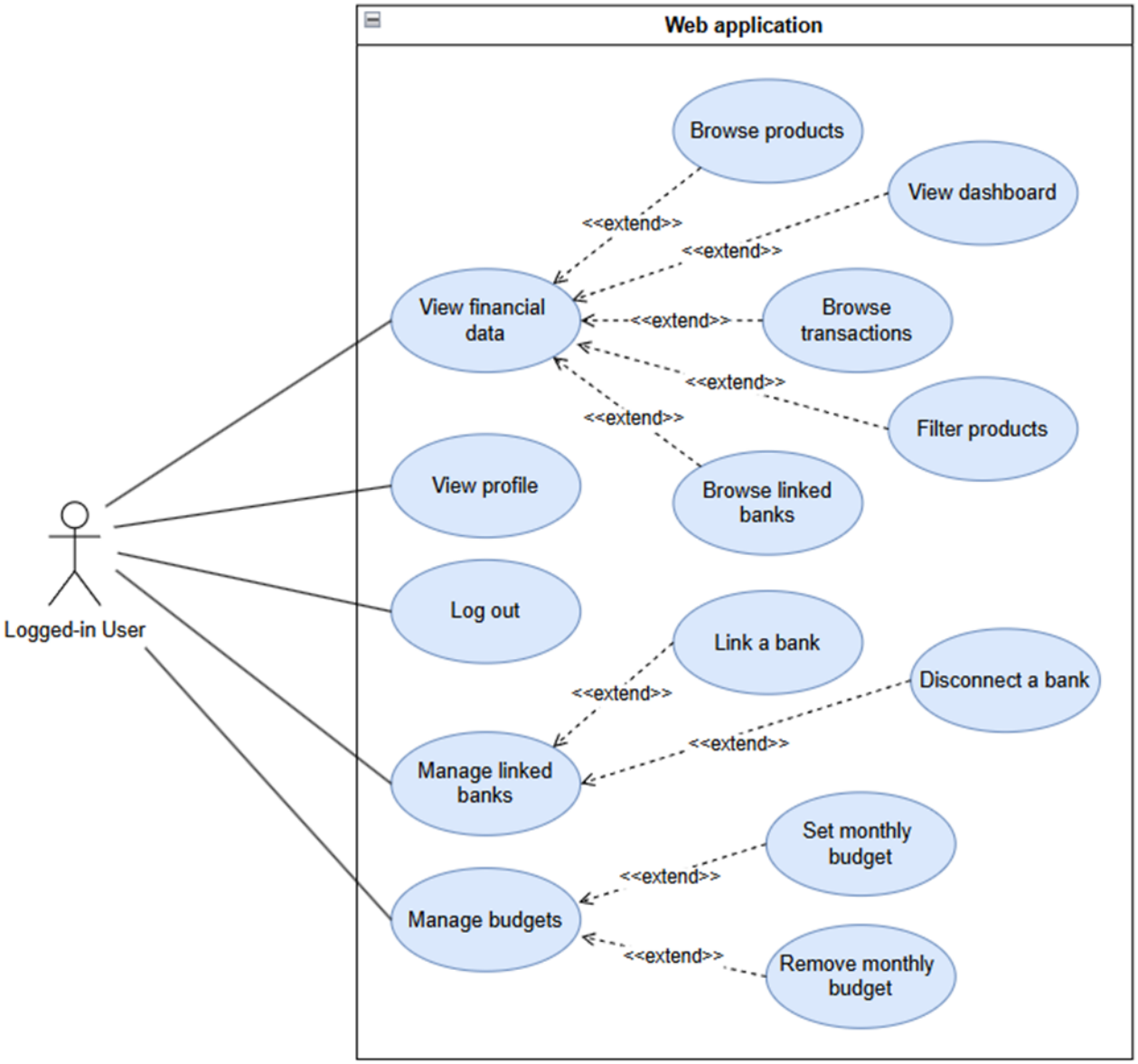

Figure 4.2. Use Cases — Logged-in User

This diagram illustrates the set of interaction scenarios available to a user who has successfully authenticated into the system, designated as the `Logged-in User` actor. Within the web application boundary, this actor maintains direct associations with five primary use cases. These base use cases are further modified through the specific extension relationship `<<extend>>` to encompass the full suite of functional requirements.

The primary use cases and their corresponding extensions implement the system's core capabilities:

- **View financial data** — This use case serves as the central hub for financial visualization and implements functional requirements from section 4.1.5. It is extended by specialized actions allowing the user to view the dashboard (FR12), browse transactions (FR14), browse products (FR15), browse linked banks, and filter products (FR21).
- **View profile** — This use case implements FR17 from section 4.1.5, enabling the user to access and review their personal account information.

- **Log out** — This use case implements FR4 from section 4.1.2, fulfilling the requirement to securely terminate the active user session.
- **Manage linked banks** — This use case represents the primary interface for banking integration and implements functional requirements from section 4.1.3. It is extended by specific actions to link a bank (FR5) and disconnect a bank (FR7).
- **Manage budgets** — This use case represents the financial planning functionalities and implements functional requirements from section 4.1.9. It is extended by specific actions to set the monthly budget (FR22) and remove the monthly budget.

4.4 Process Analysis

Process analysis describes the key workflows supported by the application and explains how individual components interact during typical operations. These processes focus on user interactions with external banking systems, internal data processing, and system responses. The aim is to clarify how functional requirements are realized in practice and to identify critical integration and security steps.

4.4.1 Connecting a Bank Account

The process of connecting a bank account enables the user to grant the application access to their financial data through an external banking API.

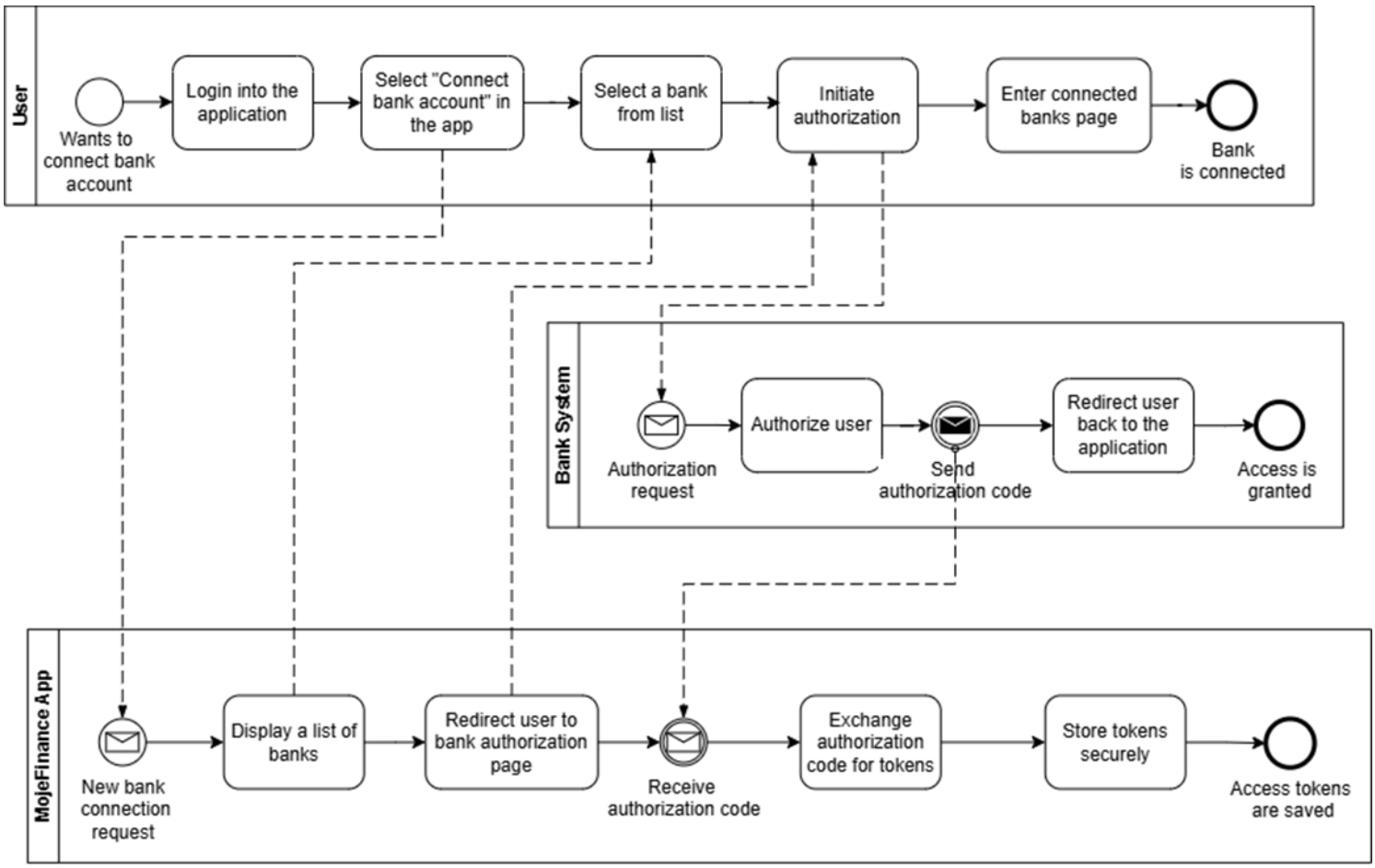

Figure 4.3. Process Diagram — Connecting a Bank Account

The process begins when a logged-in user selects a bank from the list of supported institutions within the application. After initiating the connection, the backend redirects the user to the selected bank's authentication page. At this stage, the user authenticates directly with their bank using the bank's own security mechanisms, such as two-factor authentication or biometric verification. The application itself never accesses or stores the user's banking credentials.

Once authentication is completed, the bank presents a consent screen where the user explicitly approves access to specific data scopes, typically including account information, balances, and transaction history. After consent is granted, the bank redirects the user back to the application along with an authorization code.

The backend service then exchanges this authorization code for an access token using a secure OAuth2 communication channel. The obtained tokens are securely stored in encrypted form and associated with the user's bank connection. The application uses these tokens to retrieve account and transaction data via the bank's API.

Following successful authorization, the system performs an initial synchronization, during which it retrieves the list of available accounts and their associated data. Retrieved accounts are normalized into the internal domain model and cached. Accounts become visible to the user in the application dashboard.

Chapter 5
Technology Analysis

This chapter analyzes the technologies selected for the development of the web application. The goal is to justify the chosen technology stack based on performance, maintainability, scalability, integration capabilities, and long-term sustainability.

5.1 Backend Development

5.1.1 Framework

The **Spring Boot** framework offers standardized approaches to dependency injection, configuration management, and modular application design. While lightweight micro-frameworks like Express.js or Flask offer faster initial setup times, Spring Boot excels in complex enterprise environments. For a financial application handling sensitive user data, strict typing and long-term maintainability over rapid prototyping are essential.

5.1.2 Programming Language

Java 21 was chosen as the primary programming language. In contrast to Python or JavaScript, Java provides strong static typing and relies on the highly optimized Java Virtual Machine. Furthermore, the release of Java 21 introduces virtual threads. This feature significantly improves concurrent request handling without requiring the complex, error-prone asynchronous syntax found in alternative languages, making it an exceptionally reliable choice for a high-throughput banking system.

5.1.3 Database

PostgreSQL was selected as the primary relational database management system. While NoSQL databases like MongoDB offer a flexible schema, a personal finance application requires strict relational integrity to link users, accounts, and thousands of transactions accurately. Compared to alternative relational databases like MySQL, PostgreSQL offers superior support for complex analytical queries and advanced indexing.

5.1.4 Caching

Redis serves as the in-memory data store to accelerate access to frequently requested information. External bank APIs heavily restrict request rates and regularly experience latency during peak hours. Relying solely on live API calls creates an unacceptable user experience. Compared to simpler local caching solutions like Memcached, Redis supports complex data structures and allows for highly precise control over cache expiration policies. Storing temporary values, such as session details and recent account summaries, reduces external API calls and minimizes database read operations.

5.1.5 Resilience

Integrating multiple third-party banking APIs introduces a high risk of network failures. Standard error handling is insufficient for external service outages. **Resilience4j** was integrated to implement robust fault-tolerance mechanisms, replacing older, deprecated libraries like Netflix Hystrix. It provides essential patterns such as circuit breakers, automated retries, and rate limiting. By isolating failing external services and preventing cascading system failures, Resilience4j ensures the primary application remains responsive during temporary outages of specific bank providers.

5.1.6 Messaging Configuration

Feign Client ensures communication with external banking APIs. Interacting with external endpoints traditionally requires writing repetitive boilerplate code using standard HTTP clients like RestTemplate or WebClient. Feign resolves this complexity by acting as a declarative web service client. Developers only need to define interfaces and annotations, and Feign automatically generates the underlying implementation. This approach significantly simplifies the codebase, making it highly maintainable when connecting to numerous financial institutions.

5.1.7 Identity Provider

Keycloak is used as the Identity Provider to handle user authentication and authorization. Developing a custom security module from scratch introduces significant vulnerabilities. Keycloak provides an open-source Identity and Access Management solution that fully complies with modern security protocols like OAuth 2.0 and OpenID Connect. Keycloak is entirely self-hostable, ensuring that all sensitive user credentials and access tokens remain strictly within the controlled infrastructure, maximizing data sovereignty and privacy.

5.1.8 Conclusion

The selected backend technologies form a reliable and scalable architecture designed specifically for the demands of a financial application. Java 21 and Spring Boot provide an enterprise-grade foundation emphasizing security and long-term maintainability. PostgreSQL guarantees strict data integrity, while Keycloak centralizes and secures user identity management. Concurrently, the combination of Redis and Resilience4j successfully mitigates the latency and reliability risks associated with third-party banking APIs. Finally, Feign Client streamlines external integrations. These specialized tools collectively ensure the system remains resilient, fast, and highly extensible.

Selected technologies are summarized in the following table.

Category	Technology
Framework	Spring Boot
Programming Language	Java 21
Database	PostgreSQL
Caching	Redis
Resilience	Resilience4j
Messaging Configuration	Feign Client

Table 5.1. Selected technologies for application backend development

5.2 Frontend Development

5.2.1 Framework

React is a JavaScript library for building user interfaces, created and maintained by Meta (Facebook). It is currently one of the most popular and widely adopted frontend technologies worldwide. React organizes the interface into reusable components (buttons, forms, graphs, cards, charts). This is ideal for finance dashboards, which require modular widgets. Additionally, React uses a virtual DOM, which makes UI updates fast when data changes frequently. Moreover, it has a massive ecosystem of libraries for routing, state management, charts, authentication, and UI styling, which dramatically reduces implementation effort.

Chapter 6
System Design

This chapter describes the overall system structure, architectural decisions, and internal organization of the application. It explains how individual services are arranged, how data flows through the system, and how external banking APIs are integrated. The design focuses on modularity, scalability, and maintainability while respecting security and regulatory requirements.

6.1 Architecture

This chapter presents the architectural design of the application, serving as the foundational blueprint for the entire system. Establishing a robust and well-defined architecture is crucial for ensuring the application's scalability, maintainability, and security — especially given the sensitive nature of financial data and the requirement to integrate with multiple third-party banking APIs securely. The following sections detail the chosen architectural style, the high-level system components, the internal structuring of individual services, and the specific boundaries that separate internal logic from external integrations.

6.1.1 Architectural Style

The development of the application is based on Service-Oriented Architecture (SOA). SOA is an architectural style in which a system is built as a collection of modular, reusable, and network-accessible services. Each service represents a distinct business capability — such as user management, bank integration, data synchronization, or reporting — and exposes its functionality through standardized interfaces, typically REST endpoints.

Although SOA and microservices share conceptual similarities — such as modularization, service boundaries, and network-based communication — they differ significantly in granularity, autonomy, communication style, governance, and deployment. In SOA, services can be independent, but they often share centralized components, such as databases, message busses, or identity providers. In microservices, each service is fully autonomous, owning its own database, technology stack, and deployment pipeline. This eliminates shared dependencies but increases operational complexity.

For this application, SOA provides an optimal balance. It allows for logical separation of domains without the heavy operational overhead of managing distributed databases and complex deployment pipelines required by pure microservices. As the system evolves and the need for higher scalability or independent deployment grows, this SOA foundation can naturally be refined into a microservices architecture.

6.1.2 Architecture Diagram

The architecture diagram illustrates the high-level structure of the application, showing the interaction between the frontend, backend services, external banking APIs, and the database.

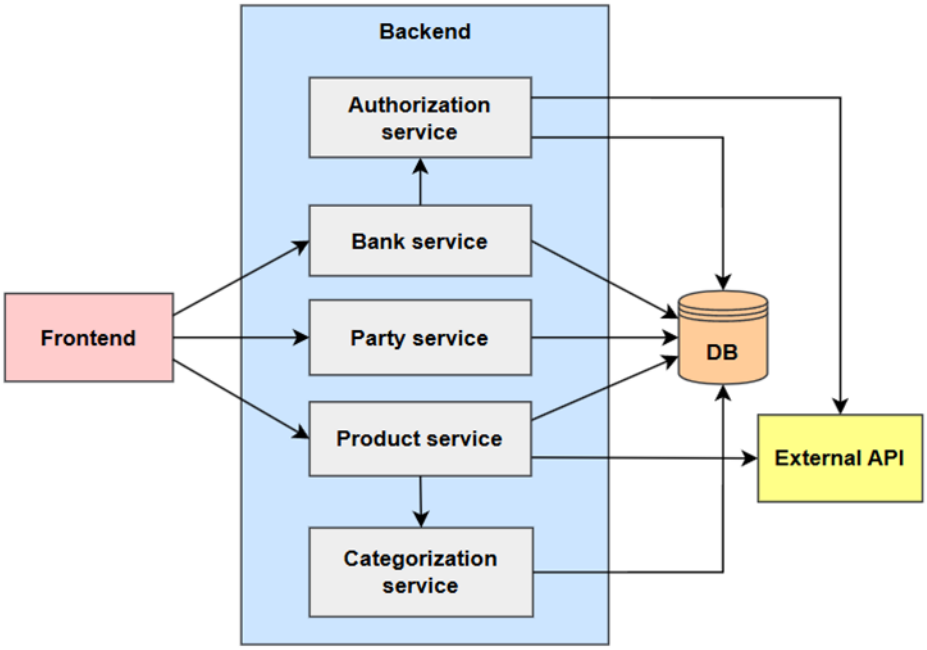

Figure 6.1. Application Architecture

The diagram in Figure 6.1 illustrates the Service-Oriented Architecture adopted for the system. The Frontend acts as the client layer, initiating requests to the backend. The Backend is decomposed into five distinct functional services: the Authorization service, the Bank service, the Party service, the Product service, and the Categorization service. Unlike a pure microservices architecture, where each service strictly owns its own database, these services share a central Database for persistence, ensuring data consistency while keeping the infrastructure manageable. The Product service also acts as a gateway to External APIs, handling the retrieval of banking data.

6.1.3 Internal Architecture

To ensure separation of concerns and maintainability, each service in the system follows a strict layered architecture composed of three main layers:

1. **Presentation Layer** — this layer exposes REST endpoints that allow clients to interact with the system. It handles incoming HTTP requests, validates input structures, and delegates execution to the domain layer. The presentation layer is strictly stateless and does not contain any business logic.
2. **Domain Layer** — this layer encapsulates the core business logic and workflows. It orchestrates operations between repositories, external providers, and domain objects. It is responsible for authorization checks, data transformation, validation rules, and transaction handling. It also contains domain entities and value objects representing the system's primary business concepts.
3. **Messaging and Persistence Layer** — this layer forms the boundary between the internal business logic and external infrastructure. The Messaging component acts as an adapter for external API implementations. Since each bank may provide different endpoints, data formats, authentication flows, or rate limits, it is essential to centralize and standardize these interactions here.

In addition to external messaging, this bottom layer manages data persistence using JPA and Hibernate. It abstracts database operations and provides a clean interface for

the Domain Layer to retrieve and store domain entities without needing to understand the underlying SQL dialects or table structures.

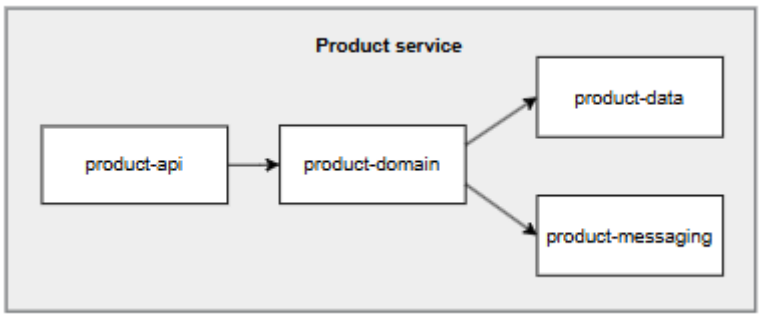

Figure 6.2. Internal Architecture

The internal architecture of the backend services, exemplified here in Figure 6.2 by the Product service, follows this strict layered pattern. The product-api component serves as the Presentation Layer, exposing REST endpoints to the client. This layer passes requests to the product-domain, which encapsulates the business logic and domain entities. The domain layer interacts with two downstream components: product-data, which handles persistence via the database, and product-messaging, which manages integrations with external banking systems.

6.1.4 Component Diagram

The component diagram illustrates the internal structure of the backend system and its interaction with the frontend, external bank APIs, and the database. It highlights how the conceptual layers described above are realized as actual software components.

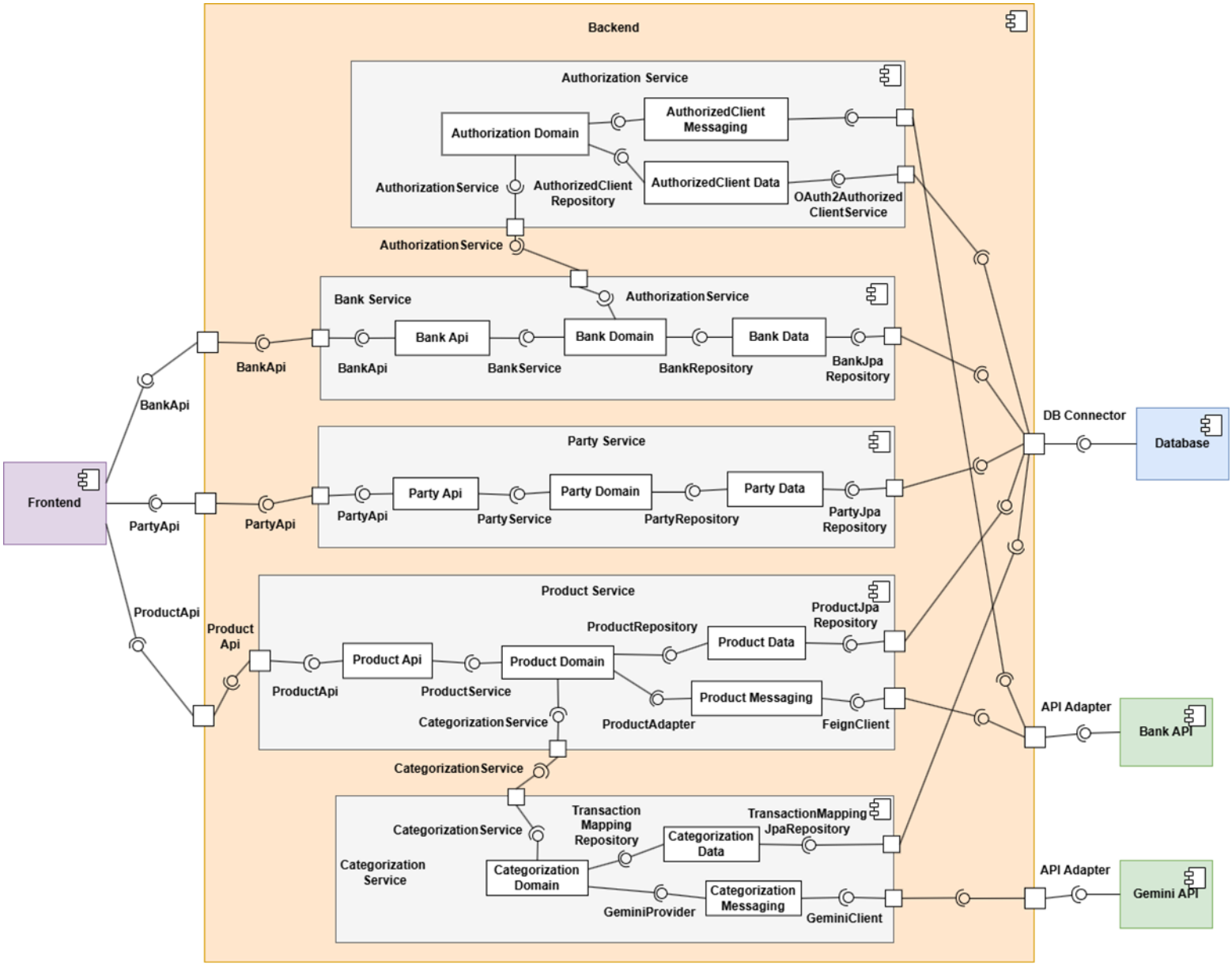

Figure 6.3. Component Diagram

The frontend communicates with the backend through exposed REST APIs. Each request is routed to the appropriate service via dedicated controllers, which represent the entry point to the backend. The diagram shows the four main backend services: Authorization Service, Bank Service, Profile Service, and Product Service. Each service is isolated within its own boundary, emphasizing modularity and preventing business logic from bleeding across domains.

Within each service, the standard layered architecture is applied. The controller layer handles incoming HTTP requests and forwards them to the application API layer. The API layer defines service-specific interfaces and orchestrates calls to the domain layer.

Persistence responsibilities are delegated to repository and data components (Authorization Data, Bank Data, Party Data, Product Data), which manage the physical database transactions. Communication with external banking systems is handled exclusively through adapter components. These APIs adapters provide a standardized

interface for interacting with varying bank APIs, effectively shielding internal services from external changes, authentication complexities, and network failures.

Overall, the component diagram demonstrates a clean separation between the presentation, application, domain, and infrastructure layers. It highlights the loose coupling between services, clear responsibility boundaries, and the high extensibility of the system.

To provide further context on the system's modularity, the core business responsibilities delegated to each backend service are defined as follows:

- **Authorization Service** — acts as the dedicated security backbone and token vault for all external banking integrations. It centrally manages OAuth2 client registrations, secure token acquisition, and the entire cryptographic lifecycle. By handling the storage, retrieval, and automated refreshing of access and refresh tokens, it ensures that sensitive credentials are strictly isolated from the rest of the application's business logic.
- **Party Service** — functions as the centralized user lifecycle manager. It completely isolates user identity and profile management from the financial domains.
- **Bank Connection Service** — manages the logical state of connected banking institutions. It abstracts the highly variable integration details of different Czech banks into a unified internal interface. Crucially, this service does not directly handle or store any security tokens. Instead, it delegates all token retrieval and refresh operations to the Authorization Service, allowing it to focus purely on orchestrating external bank communication and metadata.
- **Product Service** — serves as the core financial engine and data aggregation layer of the application. It is responsible for orchestrating financial data retrieval and normalizing data structures from various external banks into a single domain model.
- **Categorization Service** — is responsible for categorizing transactions and products based on the incoming information from the external bank API. The service uses a database table to store existing data mapping. Gemini AI API is used to categorize new data that is not yet stored in the database. This self-learning caching mechanism ensures that the system continuously expands its internal mapping dictionary, meaning future transactions with the exact same name will be categorized instantly via the database without requiring repeated, latency-heavy calls to the external AI provider.

6.2 Data design

The data design defines how information is structured, stored, and processed within the application. It focuses on the consistency, integrity, and normalization of financial data retrieved from external APIs, as well as manually created fictive data. The design ensures that real and fictive accounts can coexist within a unified data model, providing a seamless experience for the user, regardless of the data source.

6.2.1 Database Model

The database model describes the relational structure used to persist application data, user profiles, and external integration metadata. The model is designed to support both automated financial data integrations and manual fictitious data. For clarity and modularity, the overall database architecture is divided into the following specialized schemas.

6.2.1.1 Party Schema

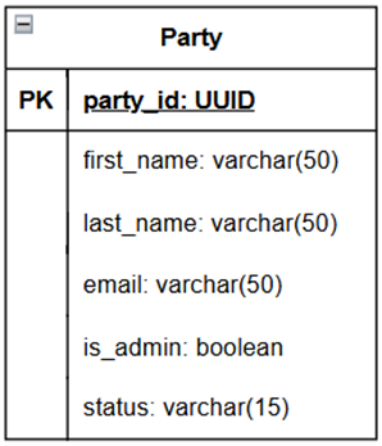

Figure 6.4. Party Schema

As illustrated in Figure 6.4, the central `Party` entity represents the system's users. This schema is responsible for storing their profile information, contact details, and administrative status, acting as the foundational root that links all other user-specific records in the application.

6.2.1.2 Authorized Client Schema

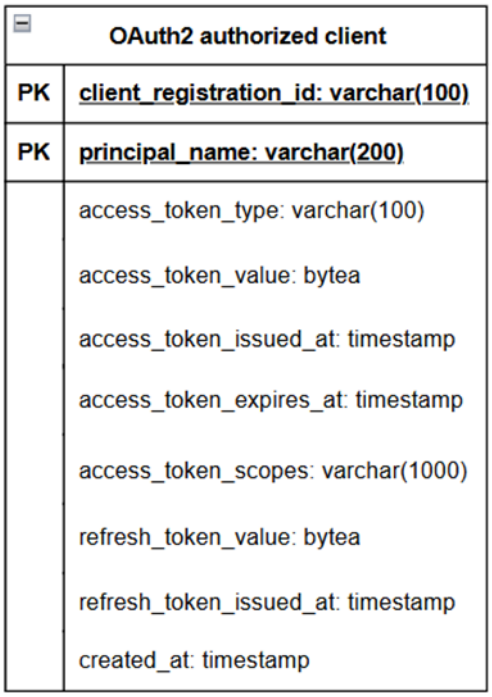

Figure 6.5. Authorized Client Schema

To handle secure communication with external banking APIs, the schema leverages Spring Security's built-in OAuth2 capabilities, depicted in Figure 6.5. The `OAuth2 authorized client` table relies on a standardized schema utilized by Spring's `OAuth2AuthorizedClientService`. Its primary responsibility is to securely persist authorized client instances across application restarts. This includes storing the

`access_token_value`, `refresh_token_value`, their respective expiration timestamps, and granted scopes. By delegating this to Spring Security, the application ensures that sensitive token lifecycle management (such as automatic token refreshing) is handled safely.

6.2.1.3 Bank Schema

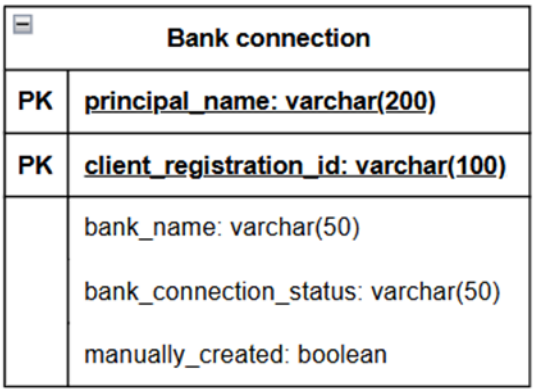

Figure 6.6. Bank Schema

`Bank connection` table, detailed in Figure 6.6, stores application-specific data about a user's linked bank. It utilizes a composite primary key — `principal_name` (representing the user) and `client_registration_id` (representing the specific bank provider). It tracks the `bank_name`, the current `bank_connection_status`, and a `manually_created` flag.

6.2.1.4 Product Schema

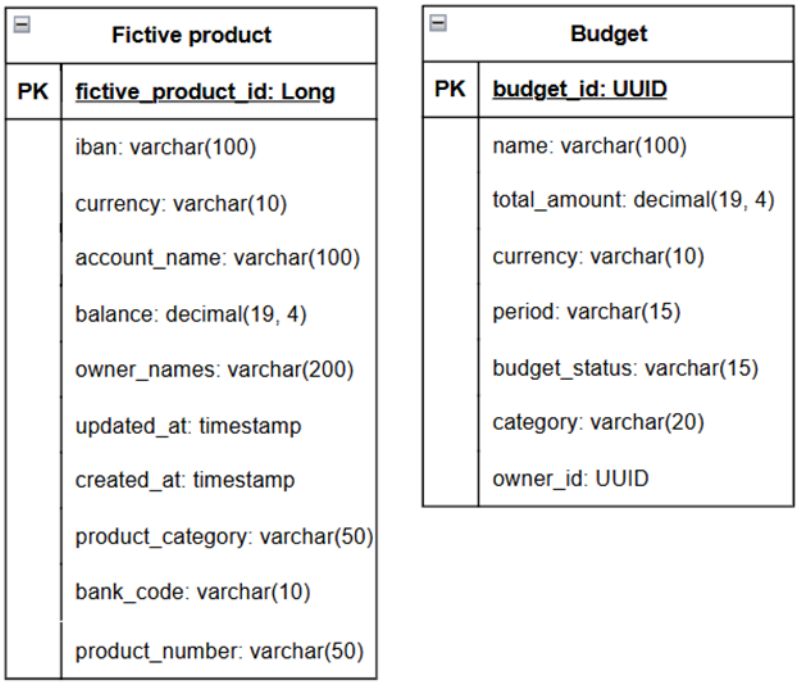

Figure 6.7. Product Schema

The product schema, presented in Figure 6.7, manages the financial accounts and balances within the system. For manual tracking, the schema uses the `Fictive product`

table. This table allows users to define custom, manually managed accounts, storing necessary details such as IBAN, currency, balance, and bank code in a single consolidated entity.

6.2.1.5 Categorization Schema

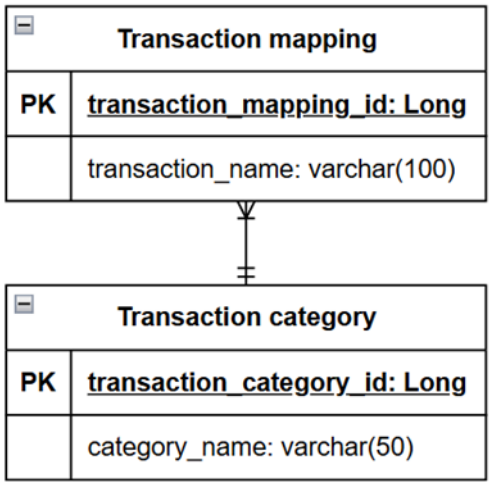

Figure 6.8. Categorization Schema

The categorization schema, illustrated in Figure 6.8, is utilized by the categorization service to store and manage how raw bank transactions are classified.

The `Transaction category` table defines the standardized categories available within the system. It contains a primary key, `transaction_category_id`, and the `category_name` representing the classification (e.g., Groceries, Salary).

The `Transaction mapping` table is responsible for linking specific raw transaction information encountered in banking data (stored in the `transaction_name` field) to these standardized categories. It uses `transaction_mapping_id` as its primary key. The schema establishes a many-to-one relationship between `Transaction mapping` and `Transaction category`, allowing multiple distinct transaction names from various bank records to be cleanly grouped under a single unified category.

6.2.2 Domain Class Model

The domain class model represents the core business entities and their relationships within the application. It abstracts technical concerns and focuses on financial concepts such as users, bank connections, accounts, and transactions. This model serves as the foundation for business logic implementation and aligns with the layered architecture used in the system.

6.2.2.1 Party Service

This section outlines the object-oriented representation of user profiles within the system. The model isolates the properties required to identify, authenticate, and manage the privileges of individuals using the platform.

The Party Service domain model focuses on user management. The primary class, `UserDomainEntity`, contains personal attributes such as first name, last name, and email. It also manages access control via the `isAdmin` flag and the status attribute. The status is defined by the `UserStatus` enumeration, which restricts users to either an ACTIVE or DEACTIVATED state.

6.2.2.2 Bank Connection Service

The class model for the Bank Connection Service maps the connection metadata required to manage integrations with third-party banking institutions.

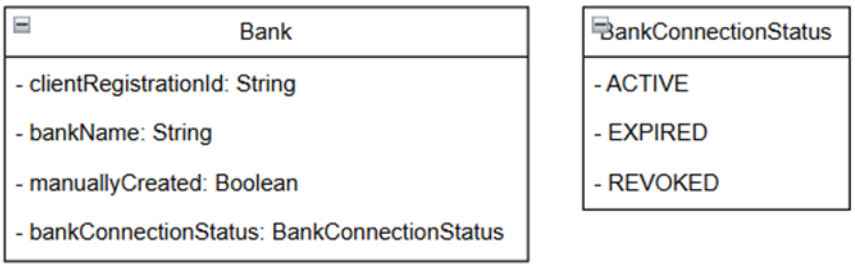

Figure 6.9. Domain Class Model - Bank Connection Service

This domain model handles the metadata and state of banking integrations rather than the security credentials. The primary entity, `Bank`, identifies the institution using a `clientRegistrationId` and records the `bankName`. It also utilizes a `manuallyCreated` flag to seamlessly distinguish between automated integrations and user-defined banks. The lifecycle of the integration is managed by the `bankConnectionStatus` attribute, utilizing the `BankConnectionStatus` enum to track if a connection is currently ACTIVE, EXPIRED, or REVOKED.

6.2.2.3 Product Service

The Product Service domain model encompasses the most data-intensive part of the application. It defines the hierarchical and relational bindings between financial products (accounts), individual transactions, data aggregations, and user-defined budgets.

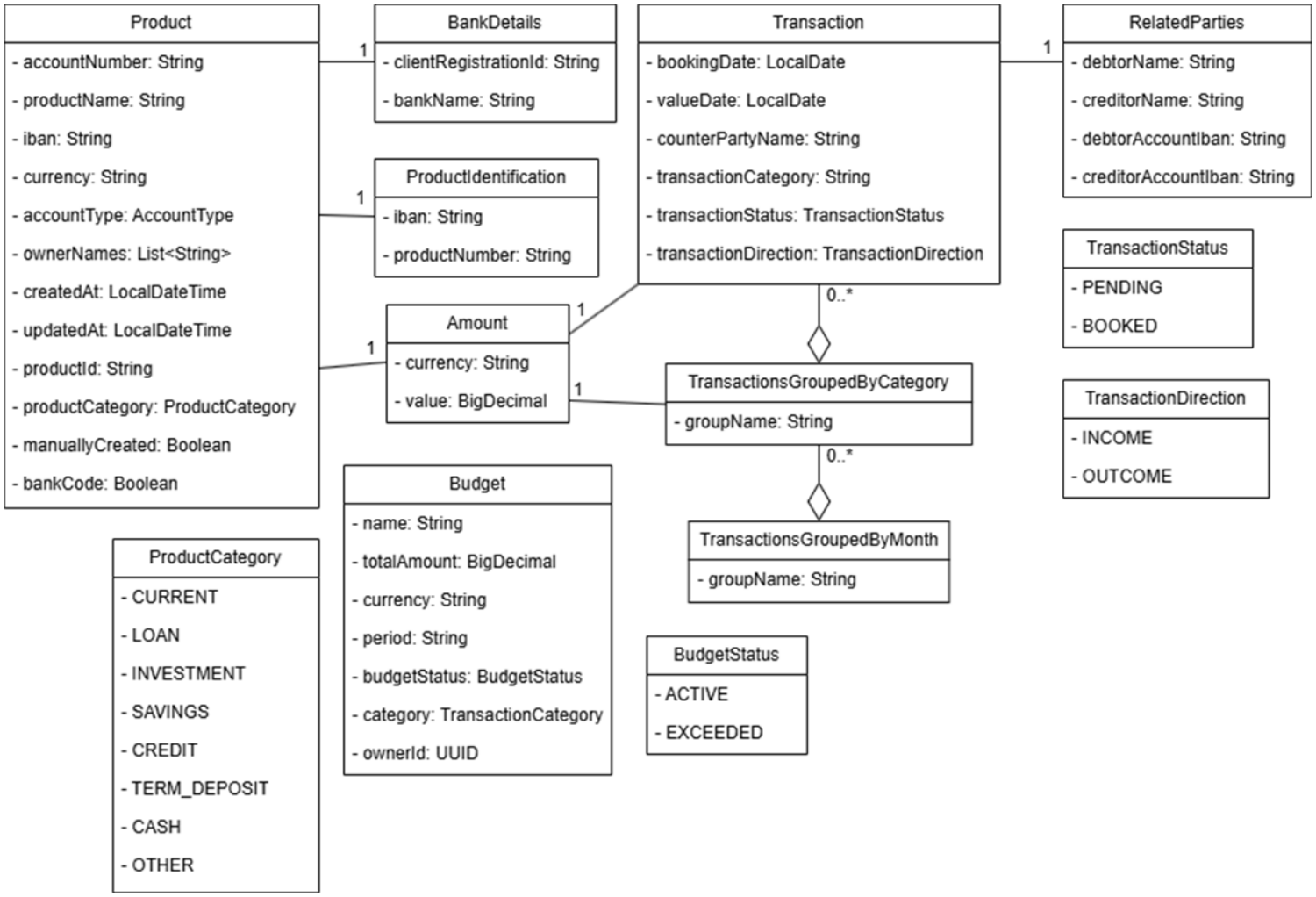

Figure 6.10. Domain Class Model - Product Service

The Product Service model forms the core financial logic of the system. It is centered around the `Product` entity, which serves as a unified representation for both synchronized bank accounts and manually created fictive accounts (distinguished by the `manuallyCreated` flag). To maintain clean data encapsulation, `Product` delegates specific attributes to associated entities like `BankDetails`, `ProductIdentification`, and `Amount`.

Individual financial movements are recorded by the `Transaction` entity. This class captures core details such as booking dates, processing states (`TransactionStatus`), and cash flow direction (`TransactionDirection`). Complex counterpart details (debtor and creditor information) are cleanly extracted into a dedicated `RelatedParties` entity.

To directly support frontend analytics and dashboard visualizations, the model introduces specialized composition structures: `TransactionsGroupedByCategory` and `TransactionsGroupedByMonth`, which hierarchically organize raw transactions for efficient reporting.

Finally, the `Budget` entity enables goal-oriented financial planning, allowing users to set monetary targets based on a specific `TransactionCategory` and track progress via the `BudgetStatus`.

6.2.2.4 Categorization Service

The Categorization Service is responsible for the automated classification of incoming bank transactions.

Figure 6.11. Domain Class Model - Categorization Service

Chapter 7
Implementation

7.1 Security

7.1.1 Keycloak

7.2 Features

7.2.1 Dashboard

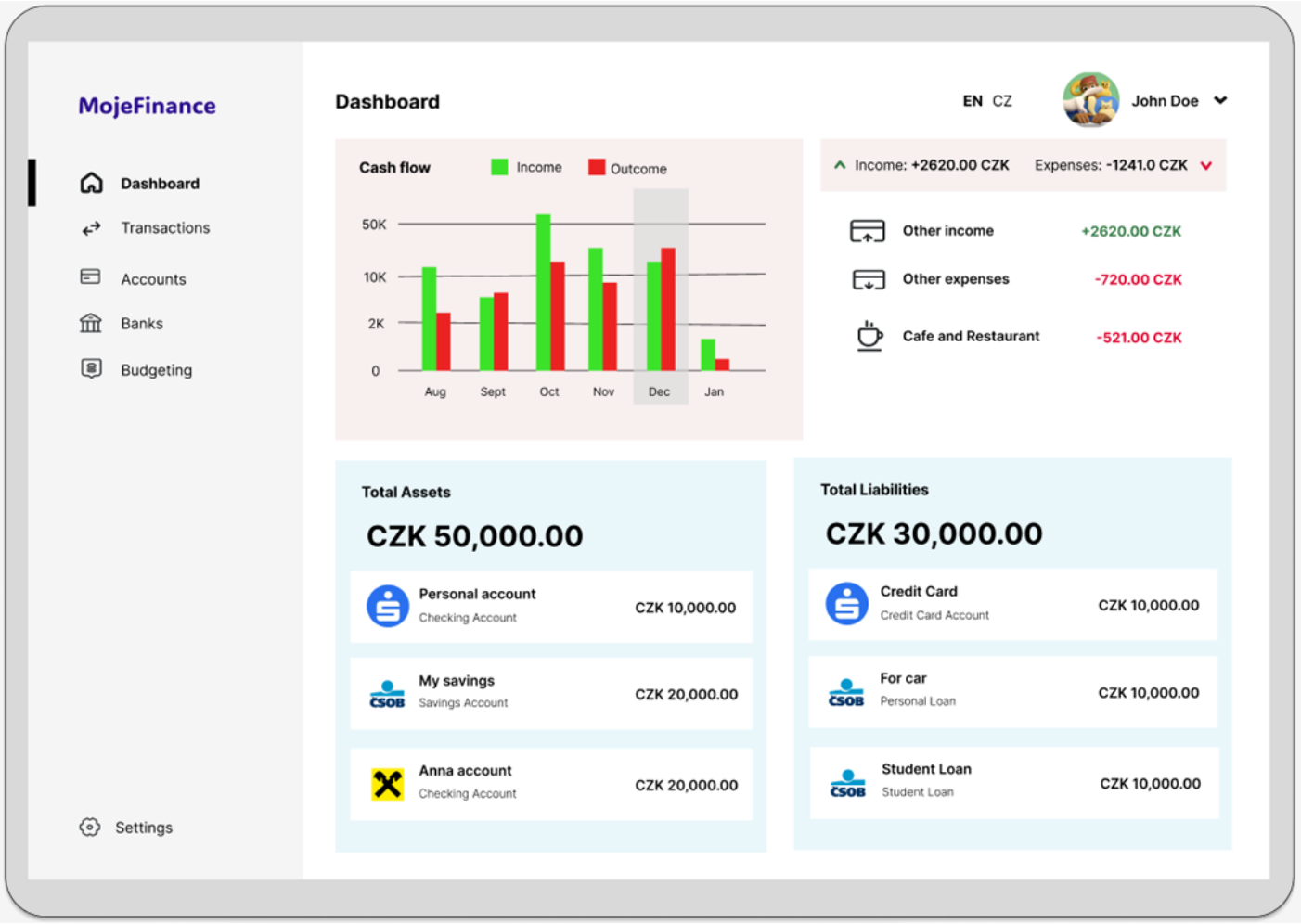

Figure 7.1. Feature — Dashboard

7.2.2 Banks Overview

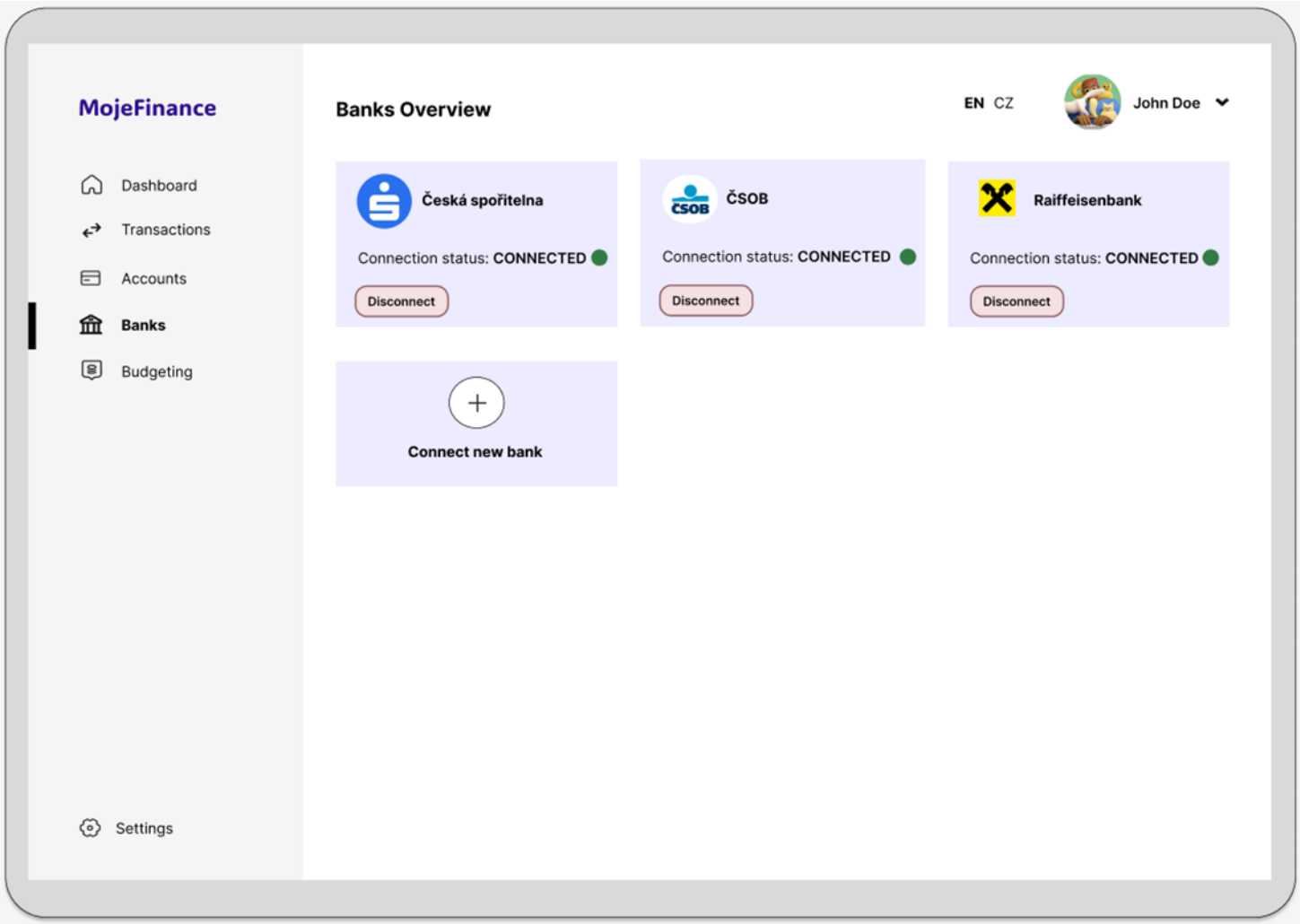

Figure 7.2. Feature — Banks Overview

7.2.3 Accounts Overview

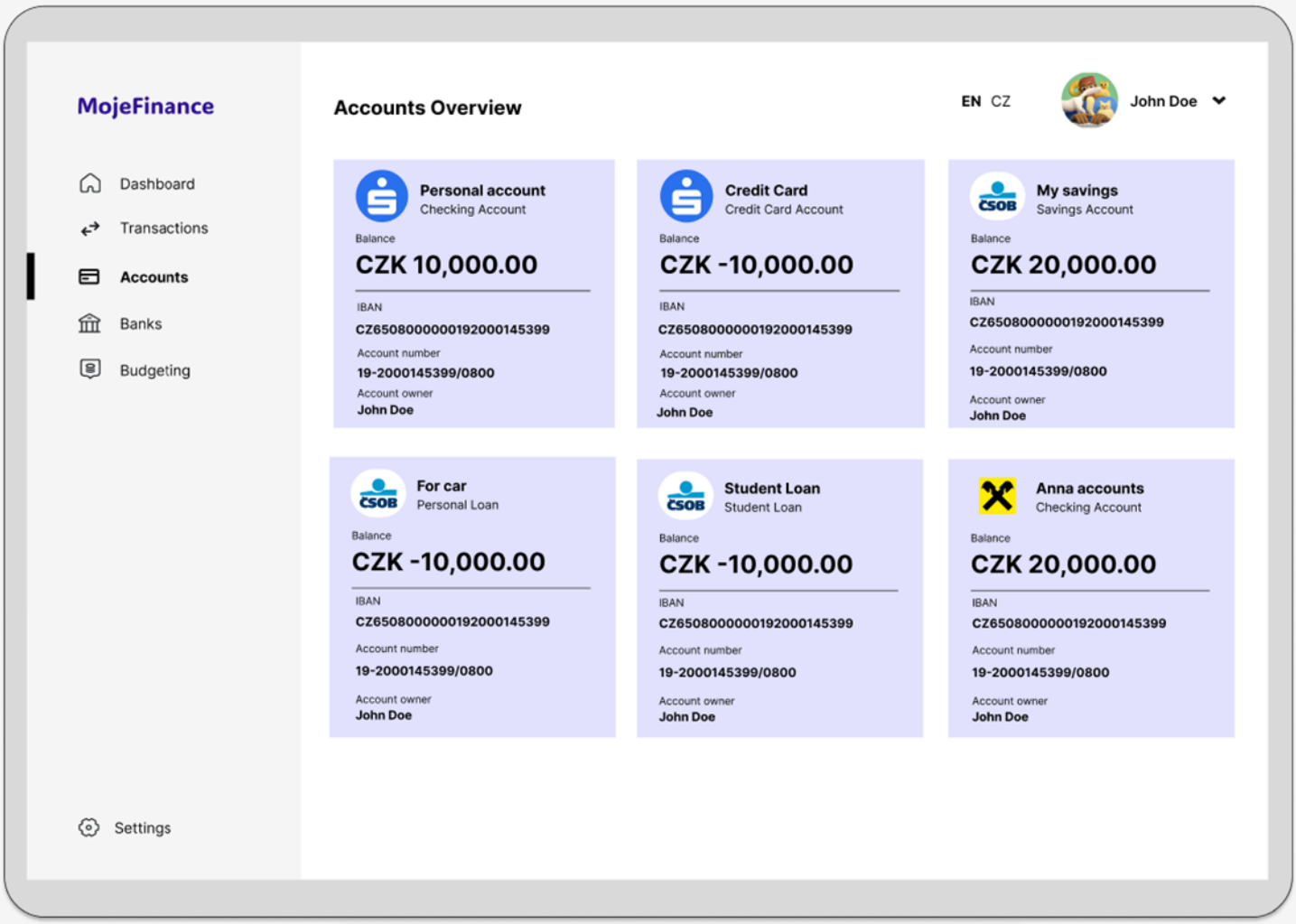

Figure 7.3. Feature — Accounts Overview

7.2.4 Transactions Overview

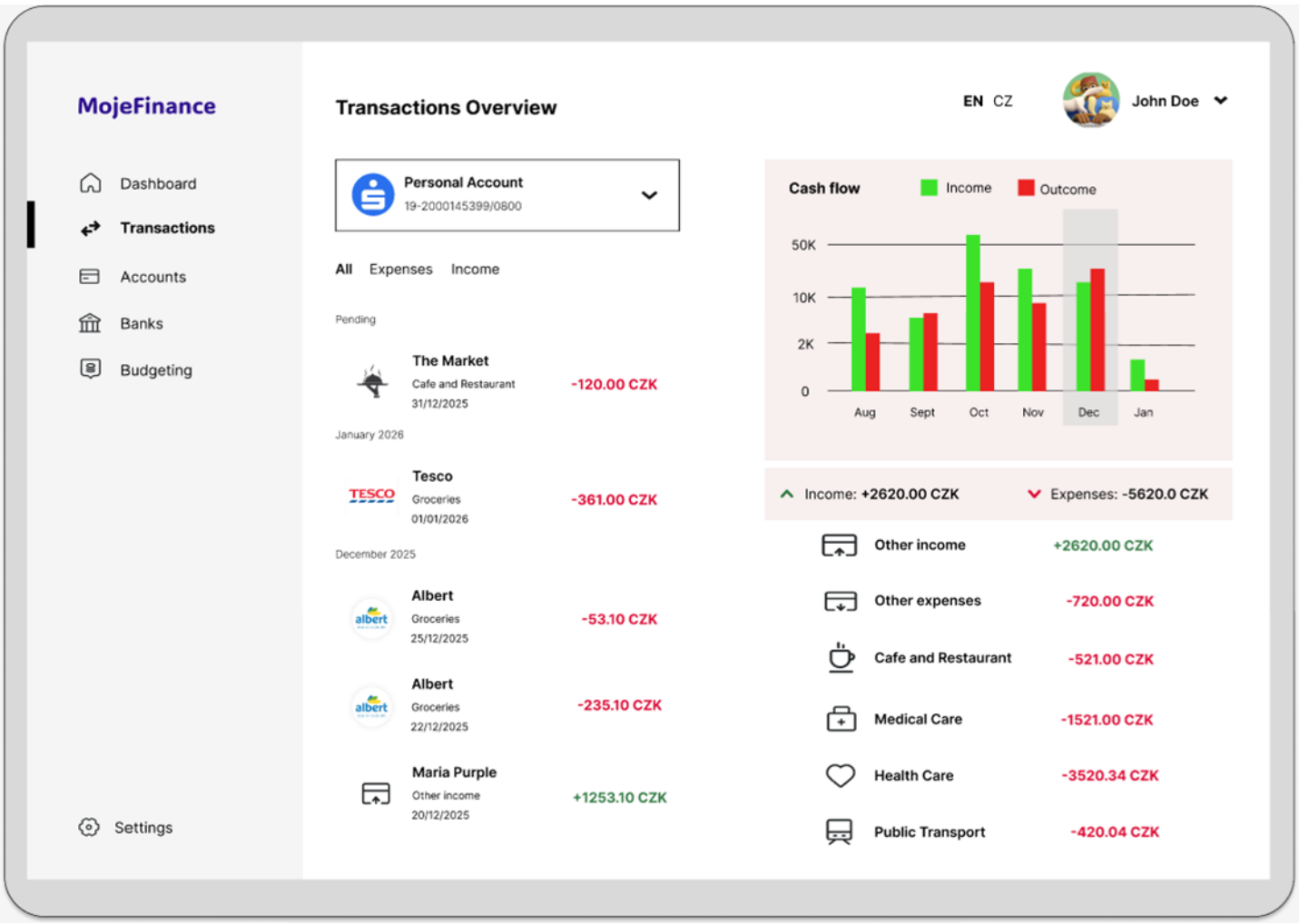

Figure 7.4. Feature — Transactions Overview

7.2.5 Budgeting

7.3 Caching and Resilience

7.4 Deployment

7.4.1 CI Pipeline

7.4.2 Docker

7.5 Error Handling

Chapter 8
Conclusion

This project demonstrates the design of a secure and user-friendly web application for managing personal bank accounts. Through analysis of existing finance platforms and Czech banking APIs, a solution was proposed that aggregates financial data, supports budgeting and visualization, and respects PSD2 and GDPR requirements. The chosen service-oriented architecture and technology stack ensure scalability, maintainability, and resilience. Although production access to most bank APIs is limited by regulatory constraints, the application remains functional through sandbox integrations and fictive accounts. The project provides a solid foundation for further development, including potential commercial deployment or extension toward advanced financial analytics.

References

[1] Vít Zýka. *FELthesis – LaTeX templates for thesis on CTU FEL*. 2012. `http://zyka.net/felthesis/felthesis.zip`.

[2] *Grafický manuál identity Českého vysokého učení technického v Praze*. Kristina Fišerová. `https://www.cvut.cz/logo-a-graficky-manual`.

Appendix A
Zadání práce

Develop a web-based application to help private users manage their multiple personal banking accounts. The application should allow users to monitor accounts from various Czech financial institutions. It should aggregate multiple accounts into a unified dashboard, allow tracking of transactions from each account, and support budgeting. Research current solutions and design the basic functionalities of the proposed application. The developed application should include functionality for displaying bank accounts, transactions, and budgets. Identify a suitable framework for the frontend and backend of your application, and justify your selection of technologies. Create a set of scenarios to verify the functionality of both individual parts and the overall application. Based on the results, evaluate the application's current state and suggest additional functionality or other improvements.

Appendix B
Zkratky a symboly

Tento text je až na výjimky převzat z [1].

B.1 Zkratky

Jako příklad pro popis zkratek poslouží pojmy ze světa TEXu.

TEX — Program na přípravu elektronické sazby vysoké kvality vytvořený Donaldem Knuthem. Program zahrnuje interpret makrojazyka. Název programu se vyslovuje „tech“.

LuaTEX — Rozšířený program TEX o možnost užití Unicode fontů a programovacího jazyka Lua.

METAFONT — Program a makro jazyk pro generování fontů z vektorového do bitmapového formátu vytvořený Donaldem Knuthem.

METAPOST — Program generující vektorovou grafiku založený na METAFONTu vytvořený Johnem Hobby.

plainTEX — Originální TEXový formát (rošíření na úrovni makrojazyka). Je součástí každé distribuce TEXu a je vytvořen Donaldem Knuthem.

OpTEX — TEXový formát rozšiřující plainTEX s využitím maker OPmac pro LuaTEX.

$\mathcal{CS}$plain — TEXový formát rozšiřující plainTEX o možnosti sazby v českém a slovenském jazyce vytvořený Petrem Olšákem.

LATEX — Nejznámější TEXový formát (rozšíření na úrovni makrojazyka) vytvořený Leslie Lamportem. Existuje obludné množství různých balíčků, které pomocí makrojazyka TEXu dále rozšiřují výchozí možnosti LATEXu. Rozličné uživatelské požadavky jsou nejčastěji řešeny použitím vhodného balíčku.

OPmac — Olšákova PlainTEXová makra nabízející uživatelům plainTEXu podobné možnosti, jako LATEX, ovšem přímočařeji a jednodušeji.

ConTEXt — Typografický systém vystavěný na LuaTEXu a na předpřipravených makro souborech vytvořený týmem v čele s Hansem Hagenem. Rozličné uživatelské požadavky jsou nastavovány pomocí přiřazení hodnot klíčovým slovům společně s možností TEXového, METAPOSTího a Lua programování.

B.2 Symboly

π — Konečná verze TEXu zmíněna v Knuthově TEXtamentu.

e — Konečná verze METAFONTu.

2ε — Současná verze LATEXu používaná od roku 1994. Počítá se s ní jako s přechodnou verzí mezi původní Lamportovou verzí LATEX 2.09 a cílovou verzí LATEX 3. Tento přechodný stav už trvá 27 let.

B.3 Soubory, které jsou součástí CTUstyle3

`ctustyle3.tex`	TEXová makra implementující šablonu ve verzi 3 (pro OpTEX).
	Logo ČVUT podle[2] v modré a černé variantě.
`Technika-Regular.otf`	Metrika a kresby písma **Technika-Regular**.
`Technika-Italic.otf`	Metrika a kresby písma ***Technika-Italic***.
`Technika-Book.otf`	Metrika a kresby písma Technika-Book.
`Technika-BookItalic.otf`	Metrika a kresby písma *Technika-BookItalic*.
`Technika-Bold.otf`	Metrika a kresby písma **Technika-Bold**.
`Technika-BoldItalic.otf`	Metrika a kresby písma ***Technika-BoldItalic***.
`ctustyle-doc.tex`	Hlavní zdrojový soubor tohoto dokumentu.
`uvod.tex, popis.tex, prilohy.tex`	Zdrojové soubory čtené z `ctustyle-doc.tex` s jednotlivými kapitolami tohoto dokumentu.
`cmelak1.jpg`	Obrázek použitý v ukázce, jak vložit obrázek.
`mybase.bib`	Údaje použité pro generování seznamu literatury.
`ctustyle-doc.pdf`	Tento dokument.
`slides.tex`	Zdrojový text ilustrující použití CTUslides.
`slides.pdf`	Výstupní ukázka ilustrující použití CTUslides včetně návodu k použití.